Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу «Операционные системы»

Группа: М8О-218БВ-25
Студент: Руснак П.Н.
Преподаватель: Бахарев В.Д.
Оценка: _____
Дата: _____

Москва, 2026

Постановка задачи

Вариант 15.

Родительский процесс создаёт дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись. Стандартный поток ввода дочернего процесса перенаправлен на pipe1, стандартный поток вывода — в File. Родительский и дочерний процесс должны быть представлены разными программами.

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child проверяет строки на валидность правилу. Если строка соответствует правилу, то она выводится в стандартный поток вывода дочернего процесса, иначе в pipe2 выводится информация об ошибке. Родительский процесс полученные от child ошибки выводит в стандартный поток вывода.

Правило проверки: строка должна начинаться с заглавной буквы (латинской A–Z).

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void);` – создаёт дочерний процесс (копию вызывающего).
- `int execv(const char *path, char *const argv[]);` – заменяет программу в процессе на исполняемый файл path; открытые дескрипторы сохраняются.
- `int pipe(int fd[2]);` – создаёт неименованный канал: fd[0] для чтения, fd[1] для записи.
- `int dup2(int oldfd, int newfd);` – делает дескриптор newfd копией oldfd; используется для перенаправления стандартных потоков.
- `int open(const char *path, int flags, mode_t mode);` – открывает (создаёт) файл и возвращает дескриптор.
- `int close(int fd);` – закрывает дескриптор.
- `ssize_t read(int fd, void *buf, size_t count);` – читает до count байт из дескриптора; 0 означает конец данных (EOF).
- `ssize_t write(int fd, const void *buf, size_t count);` – записывает до count байт в дескриптор.
- `int poll(struct pollfd *fds, nfds_t n, int timeout);` – ожидает событий сразу на нескольких дескрипторах.
- `pid_t waitpid(pid_t pid, int *status, int options);` – ожидает завершения дочернего процесса и получает его статус.
- `ssize_t readlink(const char *path, char *buf, size_t size);` – читает путь символической ссылки (/proc/self/exe — путь к своей программе).

– `sighandler_t signal(int sig, sighandler_t handler);` – задаёт реакцию на сигнал (SIGPIPE игнорируется).
– `void _exit(int status);` – немедленно завершает процесс.

Решение состоит из двух программ: parent (родитель) и child (дочерний процесс), общие функции вывода вынесены в common.h. Стандартная библиотека stdio.h не используется, весь ввод-вывод выполняется системными вызовами read и write.

Алгоритм работы:

1. parent определяет путь к программе child через /proc/self/exe, читает первую строку (имя файла) и открывает файл на запись (O_WRONLY | O_CREAT | O_TRUNC).
2. Создаются два канала: pipe1 (parent → child) и pipe2 (child → parent).
3. Вызывается fork(). В дочернем процессе dup2 перенаправляет stdin на pipe1, stdout — в файл, stderr — в pipe2; лишние дескрипторы закрываются, после чего execv загружает программу child.
4. parent закрывает ненужные концы каналов и с помощью poll одновременно следит за вводом пользователя, свободным местом в pipe1 и данными в pipe2: строки пересылаются в pipe1, сообщения об ошибках из pipe2 печатаются в stdout.
5. child читает строки из stdin (строка любой длины хранится в динамически растущем буфере). Строка с заглавной буквы записывается в stdout (файл), иначе в stderr (pipe2) выводится сообщение об ошибке.
6. По окончании ввода (EOF, Ctrl+D) parent закрывает pipe1, child получает EOF и завершается, parent получает EOF из pipe2, вызывает waitpid и проверяет код завершения дочернего процесса.

Каждый системный вызов проверяется на ошибку, причина выводится через strerror(errno). Одновременное ожидание через poll исключает взаимную блокировку (deadlock) при большом количестве ошибочных строк, когда оба канала могут заполниться.

Код программы

common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <errno.h>
#include <stddef.h>
#include <string.h>
#include <unistd.h>

static inline int write_all(int fd, const void *buf, size_t len) {
    const char *p = buf;
    while (len > 0) {
        ssize_t n = write(fd, p, len);
        if (n == -1) {
            if (errno == EINTR)
                continue;
            return -1;
        }
        p += n;
        len -= (size_t)n;
    }
    return 0;
}

static inline void write_str(int fd, const char *s) {
    (void)write_all(fd, s, strlen(s));
}

static inline void int_to_str(int value, char *out) {
    char tmp[12];
    size_t i = 0;
    long long v = value;
    int negative = v < 0;
    if (negative)
        v = -v;
    do {
        tmp[i++] = (char)('0' + v % 10);
        v /= 10;
    } while (v > 0);
    size_t j = 0;
    if (negative)
        out[j++] = '-';
    while (i > 0)
        out[j++] = tmp[--i];
    out[j] = '\0';
}

static inline void report_error(const char *prog, const char *what) {
    int saved = errno;
    write_str(STDERR_FILENO, prog);
    write_str(STDERR_FILENO, ": ");
    write_str(STDERR_FILENO, what);
    write_str(STDERR_FILENO, ": ");
    write_str(STDERR_FILENO, strerror(saved));
    write_str(STDERR_FILENO, "\n");
}

#endif
```

child.c

```
#include <errno.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "common.h"

#define PROG "child"
#define READ_CHUNK 4096

typedef struct {
    int fd;
    char buf[READ_CHUNK];
    size_t pos;
    size_t len;
} Reader;

typedef struct {
    char *data;
    size_t len;
    size_t cap;
} Line;

static int line_push(Line *line, char c) {
    if (line->len == line->cap) {
        size_t new_cap = line->cap ? line->cap * 2 : 64;
        char *p = realloc(line->data, new_cap);
        if (p == NULL)
            return -1;
        line->data = p;
        line->cap = new_cap;
    }
    line->data[line->len++] = c;
    return 0;
}

static int read_line(Reader *r, Line *line) {
    line->len = 0;
    bool got_any = false;
    for (;;) {
        if (r->pos == r->len) {
            ssize_t n = read(r->fd, r->buf, sizeof r->buf);
            if (n == -1) {
                if (errno == EINTR)
                    continue;
                return -1;
            }
            if (n == 0)
                return got_any ? 1 : 0;
            r->pos = 0;
            r->len = (size_t)n;
        }
        char c = r->buf[r->pos++];
        got_any = true;
        if (c == '\n')
            return 1;
        if (line_push(line, c) == -1)
            return -1;
```

```
    }
}

static bool starts_with_capital(const char *s, size_t len) {
    return len > 0 && s[0] >= 'A' && s[0] <= 'Z';
}

int main(void) {
    Reader reader = {.fd = STDIN_FILENO, .pos = 0, .len = 0};
    Line line = {.data = NULL, .len = 0, .cap = 0};
    int exit_code = EXIT_SUCCESS;
    int rc;

    while ((rc = read_line(&reader, &line)) == 1) {
        if (line.len > 0 && line.data[line.len - 1] == '\r')
            line.len--;

        if (starts_with_capital(line.data, line.len)) {
            if (write_all(STDOUT_FILENO, line.data, line.len) == -1 ||
                write_all(STDOUT_FILENO, "\n", 1) == -1) {
                report_error(PROG, "failed to write to output file");
                exit_code = EXIT_FAILURE;
                break;
            }
        } else {
            static const char prefix[] = "Error: line \"";
            static const char suffix[] = "\" does not start with a capital letter\n";
            if (write_all(STDERR_FILENO, prefix, sizeof prefix - 1) == -1 ||
                write_all(STDERR_FILENO, line.data, line.len) == -1 ||
                write_all(STDERR_FILENO, suffix, sizeof suffix - 1) == -1) {
                exit_code = EXIT_FAILURE;
                break;
            }
        }
    }

    if (rc == -1) {
        report_error(PROG, "failed to read input");
        exit_code = EXIT_FAILURE;
    }

    free(line.data);
    return exit_code;
}
```

parent.c

```
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <poll.h>
#include <signal.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#include "common.h"
```

```
#define PROG "parent"
#define CHILD_NAME "child"

static ssize_t read_first_line(char *buf, size_t cap) {
    size_t len = 0;
    for (;;) {
        char c;
        ssize_t n = read(STDIN_FILENO, &c, 1);
        if (n == -1) {
            if (errno == EINTR)
                continue;
            return -1;
        }
        if (n == 0 || c == '\n')
            break;
        if (len + 1 >= cap) {
            errno = ENAMETOOLONG;
            return -1;
        }
        buf[len++] = c;
    }
    if (len > 0 && buf[len - 1] == '\r')
        len--;
    buf[len] = '\0';
    return (ssize_t)len;
}

static int build_child_path(char *out, size_t cap) {
    ssize_t len = readlink("/proc/self/exe", out, cap - 1);
    if (len == -1)
        return -1;
    if ((size_t)len >= cap - 1) {
        errno = ENAMETOOLONG;
        return -1;
    }
    out[len] = '\0';

    char *slash = strrchr(out, '/');
    if (slash == NULL) {
        errno = ENOENT;
        return -1;
    }
    size_t dir_len = (size_t)(slash - out) + 1;
    size_t name_len = strlen(CHILD_NAME);
    if (dir_len + name_len + 1 > cap) {
        errno = ENAMETOOLONG;
        return -1;
    }
    memcpy(out + dir_len, CHILD_NAME, name_len + 1);
    return 0;
}

static int relay_loop(int to_child, int from_child) {
    char pending[PIPE_BUF];
    size_t pending_len = 0;
    size_t pending_off = 0;
    char buf[4096];
    bool stdin_open = true;
    int result = 0;
```

```
    while (from_child != -1) {
        struct pollfd fds[3] = {
            {.fd = from_child, .events = POLLIN, .revents = 0},
            {.fd = (stdin_open && pending_len == 0) ? STDIN_FILENO : -1,
             .events = POLLIN, .revents = 0},
            {.fd = (pending_len > 0) ? to_child : -1,
             .events = POLLOUT, .revents = 0},
        };

        if (poll(fds, 3, -1) == -1) {
            if (errno == EINTR)
                continue;
            report_error(PROG, "poll failed");
            result = -1;
            break;
        }

        if (fds[0].revents & (POLLIN | POLLHUP | POLLERR)) {
            ssize_t n = read(from_child, buf, sizeof buf);
            if (n > 0) {
                if (write_all(STDOUT_FILENO, buf, (size_t)n) == -1) {
                    report_error(PROG, "failed to write to stdout");
                    result = -1;
                }
            } else if (n == 0) {
                close(from_child);
                from_child = -1;
            } else if (errno != EINTR) {
                report_error(PROG, "failed to read from pipe2");
                result = -1;
                close(from_child);
                from_child = -1;
            }
        }

        if (fds[1].fd != -1 && (fds[1].revents & (POLLIN | POLLHUP | POLLERR))) {
            ssize_t n = read(STDIN_FILENO, pending, sizeof pending);
            if (n > 0) {
                pending_len = (size_t)n;
                pending_off = 0;
            } else if (n == 0) {
                stdin_open = false;
            } else if (errno != EINTR) {
                report_error(PROG, "failed to read from stdin");
                result = -1;
                stdin_open = false;
            }
        }

        if (fds[2].fd != -1 && (fds[2].revents & (POLLOUT | POLLERR | POLLHUP))) {
            ssize_t n = write(to_child, pending + pending_off, pending_len -
pending_off);
            if (n >= 0) {
                pending_off += (size_t)n;
                if (pending_off == pending_len)
                    pending_len = pending_off = 0;
            } else if (errno != EINTR && errno != EAGAIN) {
                report_error(PROG, "failed to write to pipe1");
                result = -1;
                pending_len = pending_off = 0;
```

```
                stdin_open = false;
                close(to_child);
                to_child = -1;
            }
        }

        if (!stdin_open && pending_len == 0 && to_child != -1) {
            close(to_child);
            to_child = -1;
        }
    }

    if (to_child != -1)
        close(to_child);
    return result;
}

int main(void) {
    char child_path[PATH_MAX];
    if (build_child_path(child_path, sizeof child_path) == -1) {
        report_error(PROG, "failed to locate child executable");
        return EXIT_FAILURE;
    }

    char filename[PATH_MAX];
    ssize_t name_len = read_first_line(filename, sizeof filename);
    if (name_len == -1) {
        report_error(PROG, "failed to read file name");
        return EXIT_FAILURE;
    }
    if (name_len == 0) {
        write_str(STDERR_FILENO, PROG ": file name is empty\n");
        return EXIT_FAILURE;
    }

    int file_fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file_fd == -1) {
        report_error(PROG, "failed to open output file");
        return EXIT_FAILURE;
    }

    int pipe1[2];
    int pipe2[2];
    if (pipe(pipe1) == -1) {
        report_error(PROG, "failed to create pipe1");
        close(file_fd);
        return EXIT_FAILURE;
    }
    if (pipe(pipe2) == -1) {
        report_error(PROG, "failed to create pipe2");
        close(file_fd);
        close(pipe1[0]);
        close(pipe1[1]);
        return EXIT_FAILURE;
    }

    if (signal(SIGPIPE, SIG_IGN) == SIG_ERR) {
        report_error(PROG, "failed to ignore SIGPIPE");
        return EXIT_FAILURE;
    }
```

```
pid_t pid = fork();
if (pid == -1) {
    report_error(PROG, "failed to fork");
    close(file_fd);
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    return EXIT_FAILURE;
}

if (pid == 0) {
    signal(SIGPIPE, SIG_DFL);

    if (dup2(pipe1[0], STDIN_FILENO) == -1 ||
        dup2(file_fd, STDOUT_FILENO) == -1 ||
        dup2(pipe2[1], STDERR_FILENO) == -1) {
        report_error(PROG " (child branch)", "dup2 failed");
        _exit(EXIT_FAILURE);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    close(file_fd);

    char *const args[] = {(char *)CHILD_NAME, NULL};
    execv(child_path, args);

    report_error(CHILD_NAME, "failed to exec");
    _exit(127);
}

close(pipe1[0]);
close(pipe2[1]);
close(file_fd);

int relay_result = relay_loop(pipe1[1], pipe2[0]);

int wstatus;
while (waitpid(pid, &wstatus, 0) == -1) {
    if (errno != EINTR) {
        report_error(PROG, "waitpid failed");
        return EXIT_FAILURE;
    }
}

bool child_ok = false;
char num[12];
if (WIFEXITED(wstatus)) {
    int code = WEXITSTATUS(wstatus);
    child_ok = (code == 0);
    if (!child_ok) {
        int_to_str(code, num);
        write_str(STDERR_FILENO, PROG ": child exited with code ");
        write_str(STDERR_FILENO, num);
        write_str(STDERR_FILENO, "\n");
    }
```

```
    } else if (WIFSIGNALED(wstatus)) {
        int_to_str(WTERMSIG(wstatus), num);
        write_str(STDERR_FILENO, PROG ": child killed by signal ");
        write_str(STDERR_FILENO, num);
        write_str(STDERR_FILENO, "\n");
    }

    return (relay_result == 0 && child_ok) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Makefile

```
CC      = gcc
CFLAGS  = -std=c11 -Wall -Wextra -Wpedantic -O2 -g -D_POSIX_C_SOURCE=200809L

all: parent child

parent: parent.c common.h
    $(CC) $(CFLAGS) -o $@ parent.c

child: child.c common.h
    $(CC) $(CFLAGS) -o $@ child.c

clean:
    rm -f parent child

.PHONY: all clean
```

Протокол работы программы

Тестирование:

Сборка:

```
$ make
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -g -D_POSIX_C_SOURCE=200809L -o
parent parent.c
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -g -D_POSIX_C_SOURCE=200809L -o
child child.c
```

Тест 1. Основная работа (ввод с клавиатуры, завершение Ctrl+D):

```
$ ./parent
result.txt
Hello world
hello world
Error: line "hello world" does not start with a capital letter
123 abc
Error: line "123 abc" does not start with a capital letter
Ok.

Error: line "" does not start with a capital letter
  Leading space
Error: line "  Leading space" does not start with a capital letter
The end
$ echo $?
0
$ cat result.txt
Hello world
Ok.
The end
```

Тест 2. Невозможно открыть файл:

```
$ ./parent
/nonexistent/dir/f.txt
parent: failed to open output file: No such file or directory
$ echo $?
1
```

Тест 3. Программа child не найдена (ошибка execv):

```
$ ./parent
a.txt
Abc
child: failed to exec: No such file or directory
parent: child exited with code 127
```

Тест 4. Нагрузочный: 400 000 строк, половина ошибочных — программа завершается без зависания, в файл записано 200 000 строк, на экран выведено 200 000 ошибок.

Вывод

В ходе лабораторной работы я освоил создание процессов в Linux с помощью fork и execv, обмен данными между процессами через неименованные каналы и перенаправление стандартных потоков через dup2.

Дочерняя программа не знает о каналах и файле: она работает только со стандартными потоками, а связывание выполняет родитель.

Основные трудности: необходимость закрывать неиспользуемые концы каналов (иначе процессы не получают EOF и зависают), разбор строк произвольной длины из потокового канала без stdio.h, а также риск взаимной блокировки при заполнении обоих каналов, который был устранён ожиданием через poll.